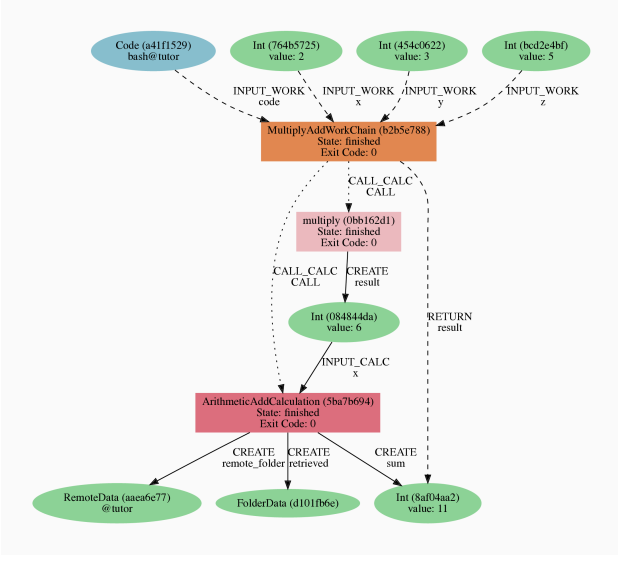
AiiDA behind the curtain

Julian Geiger · PSI

MaterialsCommons WP4 · 7 September 2026



Plenty of workflow managers

`pditommaso/awesome-pipeline`

↓

237

`meirwah/awesome-workflow-engines`

↓

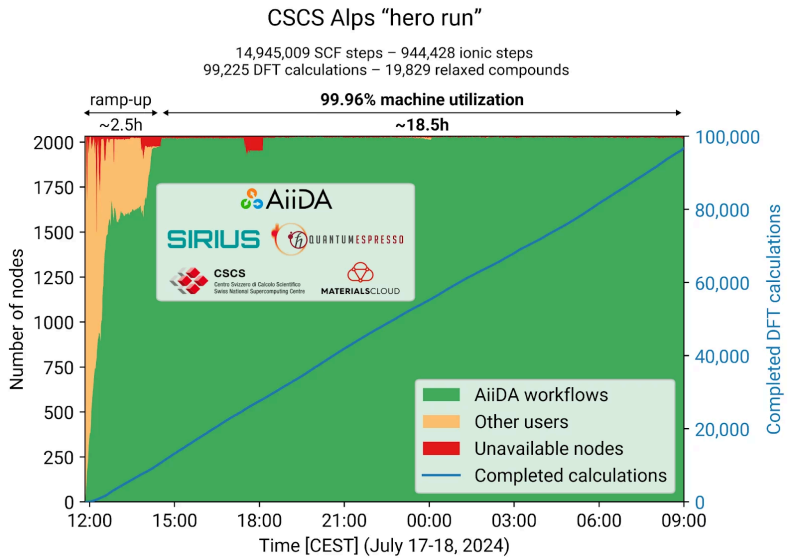
115

The difference lies in their foundations and architecture.

github.com/pditommaso/awesome-pipeline
github.com/meirwah/awesome-workflow-engines

Foundations

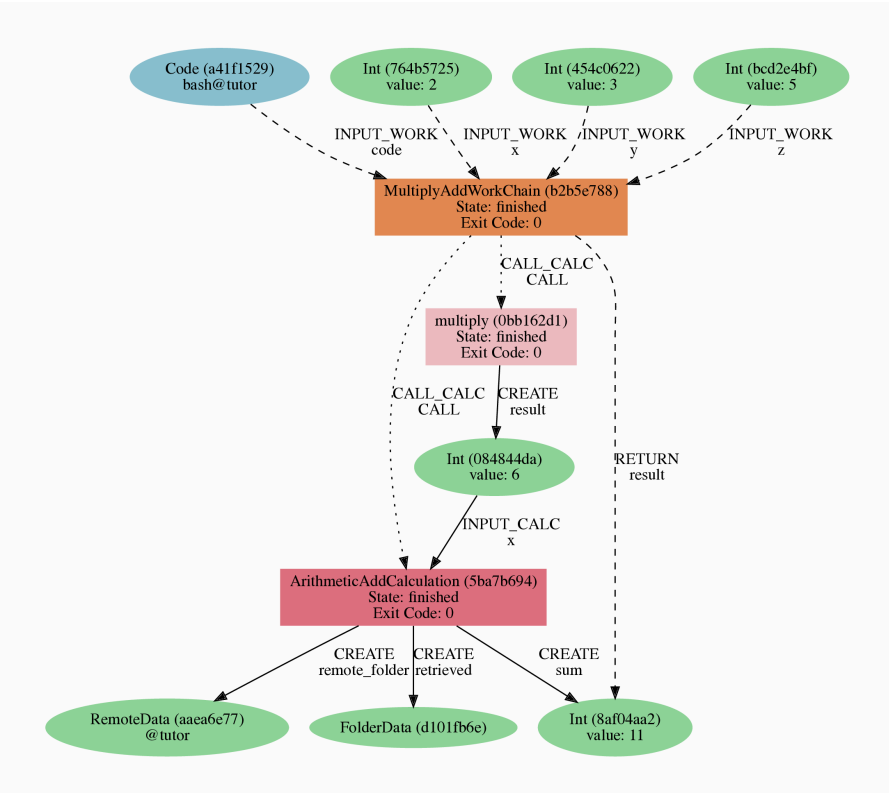
High-throughput



2033 Grace Hopper nodes, **8132 GPUs and 585,504 CPU cores**, 99.96 % utilisation for 18 h

Throughput scales with the number of daemon workers, independent Python processes, up to about one per CPU core

Provenance



- Recorded as the workflow runs: `while_`, `if_`, dynamic submission
- Immutable once stored, links validated, processes sealed
- Results traceable at any future point in time

Architecture

Source layout

```
src/aiida/
├─ brokers/
├─ calculations/
├─ cmdline/
├─ common/
├─ engine/
├─ manage/
├─ orm/
├─ parsers/
├─ plugins/
├─ repository/
├─ restapi/
├─ schedulers/
├─ sphinxext/
├─ storage/
├─ tools/
├─ transports/
└─ workflows/
```

→ **bold**: the rest of this section

Storage

- PostgreSQL / SQLite: the graph, plus each node's `attributes` and `extras` as JSONB
- the file repository, `disk-objectstore`:
  - content-addressed by SHA-256, deduplicated
  - raw files, never queried by content
- Only file-content hash crossing between both

`repository_metadata`, a column on the `Node`

```
{"o": {"aiida.in": {"k": "0fc524ff…bfeb1"},
       ".aiida": {"o": {
          "calcinfo.json": {"k": "58d7e318…f782a"}}}}}
```

`o` for `objects`, recursive; `k` for `key`, the SHA-256

the container,

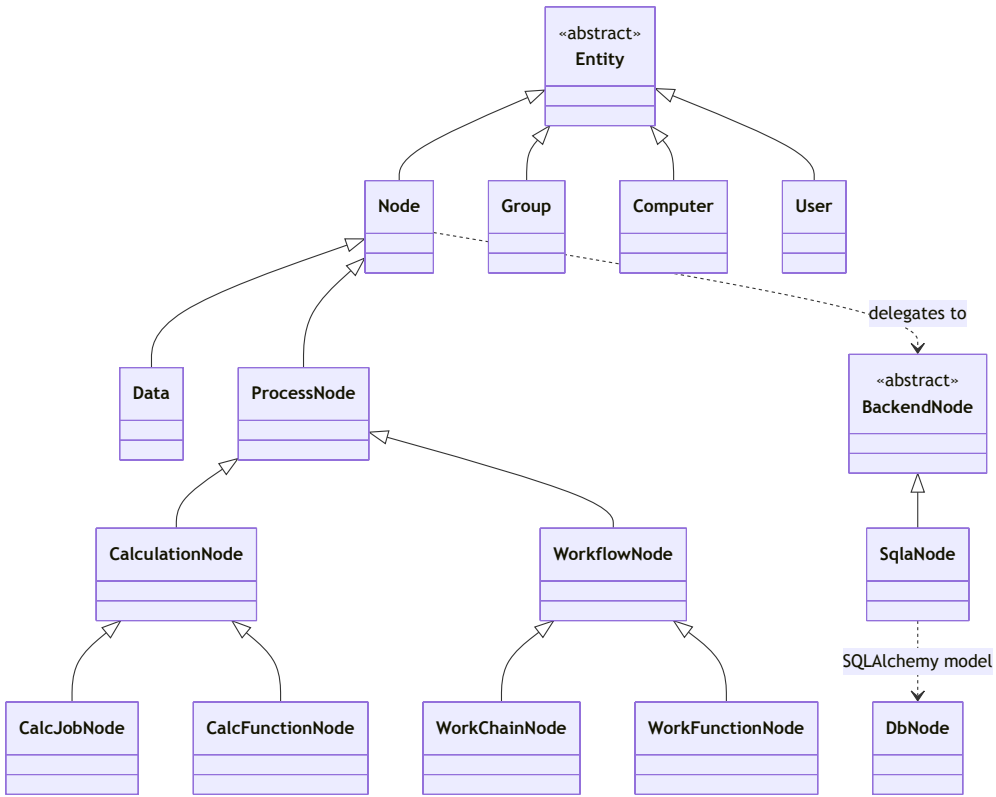
`$AIIDA_PATH/.aiida/repository/<profile>/container/`

```
loose/aa/bbcc…   one file per object
packs/0,1,2      append-only, 4 GiB target
packs.idx        SQLite: hash → pack, offset
```

one `loose` file per new object, packed by `verdi storage maintain`

github.com/aiidateam/disk-objectstore

The ORM, object-relational mapping



- Interact with DB objects as Python classes
- `Entity` : pk, uuid, `store()`
- `Node` : links, attributes, extras, file repository
- `Data` subclasses: primitives, files, folders, structures, arrays

Querying the graph

```
qb = QueryBuilder()
qb.append(
    Group,
    filters=Group.fields.label == 'mc3d/structures',
    tag='group',
)
qb.append(
    StructureData,
    with_group='group',
    tag='structure',
    project=[StructureData.fields.uuid],
)
qb.append(
    CalcJobNode,
    with_incoming='structure',
    filters=(
        (CalcJobNode.fields.process_state == 'finished')
        & (CalcJobNode.fields.exit_status == 0)
    ),
    project=['*'],
)

qb.all()
```

- Path through the graph with `append`
- Relationships: `with_group`, `with_incoming`, `with_outgoing`, `with_ancestors`, `with_descendants`
- `Class.fields.*` for filters and projections, typed and autocompleting
- `project`: what each result row carries

Submitting a workflow

```python
from aiida.engine import submit

node = submit(SomeWorkChain, **inputs)  # returns immediately
```

- One `ProcessNode` row per process
- `submit()` serialises inputs, context and state into `checkpoints`
- Daemon worker — background Python process with an event loop — picks it up
- Process rebuilt from its checkpoint, stepped to the next state
- Every `Process` is a state machine: `CREATED`, `RUNNING`, `WAITING`, `FINISHED`, `EXCEPTED`, `KILLED`
- Checkpoint rewritten at every transition: a killed daemon resumes
- `CalcJob`: the process wrapping one simulation code

The daemon and the broker

Daemon

- `verdi daemon start N`: N workers, supervised by `circus`
- One worker: one OS process, one asyncio loop
- Each AiiDA process: a coroutine on that loop

Broker

- The message layer between `submit()`, the workers, and `verdi`
- RPC: pause, play, kill one running process by its pk
- Broadcast: `state_changed`, subscribed to by waiting workchains
- RabbitMQ and ZeroMQ as available brokers

Add ZeroMQ-based message broker plugin github.com/aiidateam/aiida-core/pull/7284

`verdi`, the command line

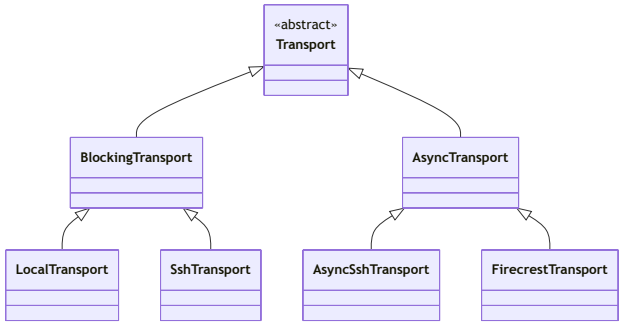
```
$ verdi status
 ✔ version:     AiiDA v2.10.0.dev0
 ✔ config:      /home/geiger_j/aiida_projects/aiida-dev/.aiida
 ✔ profile:     tutorial
 ✔ storage:     SqliteDosStorage[/home/geiger_j/aiida_projects/aiida-dev/.aiida/repository/sqlite_dos_f4ff…]: open,
 ✔ daemon:      Daemon is running with PID 830741, Broker PID 830744 [0 pending, 0 processing]
```

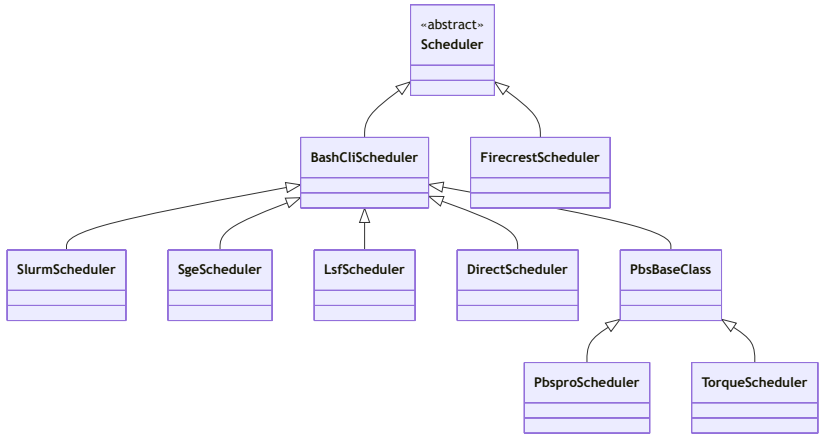
```
$ verdi process list -a
  PK  Created    Process label             ♻    Process State    Process status
----  ---------  ------------------------  ---  ---------------  ----------------
   5  5s ago     MultiplyAddWorkChain           ■ Finished [0]
   6  5s ago     multiply                       ■ Finished [0]
   8  5s ago     ArithmeticAddCalculation       ■ Finished [0]

Total results: 3

Report: ♻ Processes marked with check-mark were not run but taken from the cache.
Report: Last time an entry changed state: 4s ago (at 15:41:34 on 2026-09-06)
Report: Using 0% of the available daemon worker slots.
```

Transport and scheduler





- `Transport` and `Scheduler` : file transfer and remote commands, job submission
- `aiida-firecrest` : transport and scheduler for the FirecREST API, from CSCS
- `TransportQueue` : one shared transport per computer and user
- One `squeue` per computer per polling interval
- Exponential backoff on transport tasks, ending in a paused state

Transport: AsyncTransport plugin github.com/aiidateam/aiida-core/pull/6626

Making it simpler



No services required

```
pip install aiida-core
verdi presto
```

- PostgreSQL the only database until v2.5, now also SQLite (a file, no server)
- RabbitMQ the only broker until v2.9, now also ZeroMQ
- ZeroMQ Broker started and supervised by the daemon (RMQ runs as OS background process)
- IPC sockets, local only; workers on another host (currently) still need RabbitMQ

Released in v2.5.0 · Add the `SqliteDosStorage` *storage backend* github.com/aiidateam/aiida-core/pull/6148
Released in v2.9.0 · Add ZeroMQ-based message broker plugin github.com/aiidateam/aiida-core/pull/7284

Profile creation

- An AiiDA profile: one database, one file repository, one broker

```
# install PostgreSQL
# and RabbitMQ, create
# the database and its
# user, then describe it
verdi setup \
    --database-engine \
    --database-backend \
    --database-hostname \
    --database-port \
    --database-name \
    --database-username \
    --database-password \
    --broker-protocol \
    --broker-username \
    --broker-password \
    --broker-host \
    --broker-port \
    --broker-virtual-host \
    --repository-uri
```

```
$ verdi presto --use-zeromq
Report: Option `--use-postgres` not enabled: configuring the profile to use SQLite.
Report: `--use-zeromq` enabled: configuring the profile with ZeroMQ broker.
Report: Initialising the storage backend.
Report: Migrating to the head of the main branch
Report: Storage initialisation completed.
Success: Created new profile `presto`.
Success: Configured the localhost as a computer.
```

- `verdi quicksetup` created the database, still needed both services
- `verdi presto` : no required options since v2.6, no services since v2.9
- Full control retained: `verdi profile setup core.psql_dos`
- **progressive disclosure**

Current efforts

v3: Consolidation into core

What is moving

- `aiidateam/plumpy`, the process state machine
- The workflow construction ecosystem:
  - `aiida-shell`: cmd execution without plugins
  - `aiida-pythonjob`: execute Python on remote
  - `node-graph`: graph-based workflow API
  - `aiida-workgraph`: exec layer of `node-graph` API
- Minor upstream changes require releases
- Blurred package boundaries, and the coupling that follows
- Easier reasoning on architecture, code deduplication

Directions

- Less inheritance, more composition
- ORM and storage redesigns
- Still open to extensions via ABCs, Protocols, and entry points

Redesign ORM with Pydantic-first mindset github.com/aiidateam/aiida-core/issues/7391
v3: Storage architecture redesign github.com/aiidateam/aiida-core/issues/7510

v3: Workflow construction

`WorkChain`

```
class MultiplyAddWorkChain(WorkChain):
    @classmethod
    def define(cls, spec):
        super().define(spec)
        spec.input('x', valid_type=Int)
        spec.input('y', valid_type=Int)
        spec.input('z', valid_type=Int)
        spec.input('code', valid_type=AbstractCode)
        spec.outline(
            cls.multiply,
            cls.add,
            cls.validate_result,
            cls.result,
        )
        spec.output('result', valid_type=Int)
```

plus four methods passing data through `self.ctx`

`WorkGraph` (used for PWD)

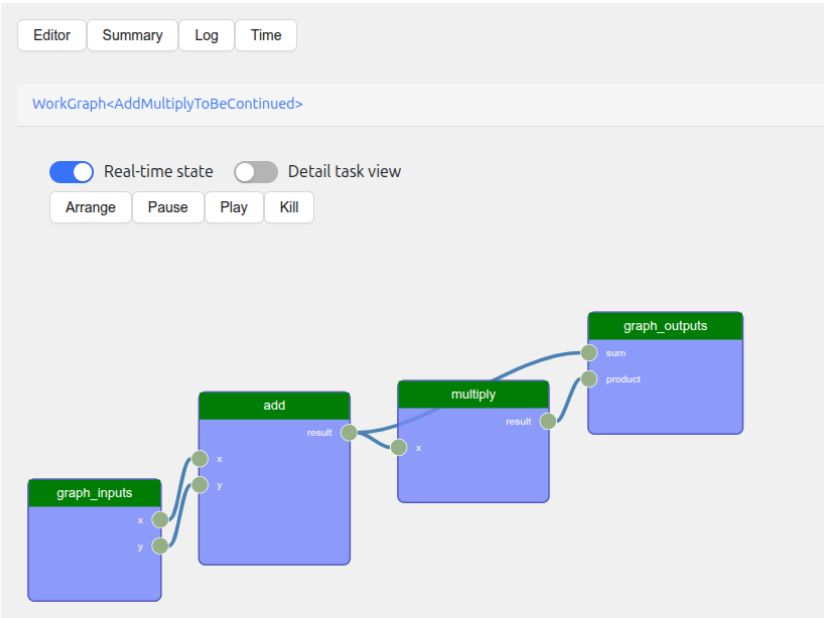
```
@task
def add(x: int, y: int) -> int:
    return x + y


@task
def multiply(x: int, y: int) -> int:
    return x * y


@task.graph
def AddMultiply(x: int, y: int, z: int) -> int:
    the_sum = add(x=x, y=y).result
    return multiply(x=the_sum, y=z).result


wg = AddMultiply.build(x=2, y=3, z=4)
wg.run()
```

v3: Workflow as an object



- `AddMultiply.build(...)` : an object, before submission
- Inspectable and editable in Python, then run
- GUI: visualize tasks, sockets, links
- `WorkChain` : nothing to inspect before it runs

v3: User and core API

- Most symbols re-exported upwards: `orm.StructureData`
- Deep paths still reachable: `aiida.orm.nodes.data.structure`
- Convention: below the second level private, unenforced
- v3: small `core` and `user` modules, importing and re-exporting explicitly
- Everything outside them internal, free to change
- Per-module migration files, to upgrade plugins automatically

AEP v3: clear public and private API github.com/aiidateam/aiida-core/issues/7410
v3: user/core API revision github.com/aiidateam/aiida-core/pull/7562

Agents

`aiida-agents`, GSoC 2026

```
src/aiida_agents/
├─ agents/    analysis · codegen · execution · planner
├─ mcp/       MCP server, read-only tools
├─ rag/       chunking, embedding, retrieval, citations
├─ sandbox/   profile copy, code runner, static guard
└─ tools/     typed tool functions
```

- Explore, inspect, diagnose, look up the docs, set up, run
- Typed tool calls against your own provenance graph
- Writes behind an explicit approval gate
- Provider agnostic, supporting local models

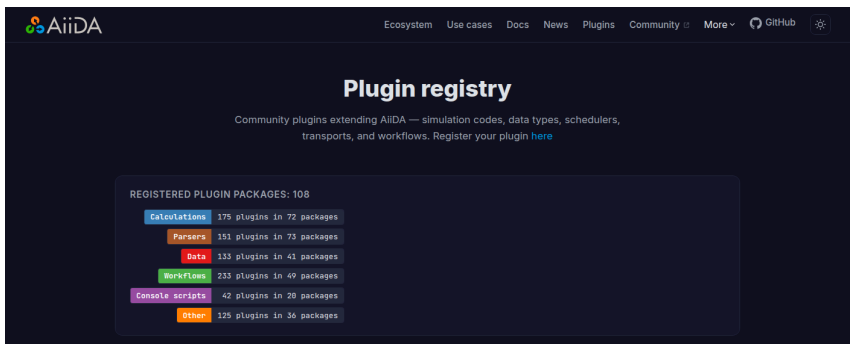
How we develop

- `AI_POLICY.md`: a responsible human author per PR
- `AGENTS.md` and 10 Claude Code skills in-repo
- CodeRabbit reviews on every non-draft PR
- Large-scale architectural changes within reach through the help of agents

Add AI_POLICY.md github.com/aiidateam/aiida-core/pull/7229
Add AGENTS.md and Claude Code skills github.com/aiidateam/aiida-core/pull/7265
Add explicit CodeRabbit configuration github.com/aiidateam/aiida-core/pull/7471

Ecosystem

Plugins and common workflows





108 plugin packages, 72 wrapping a simulation code

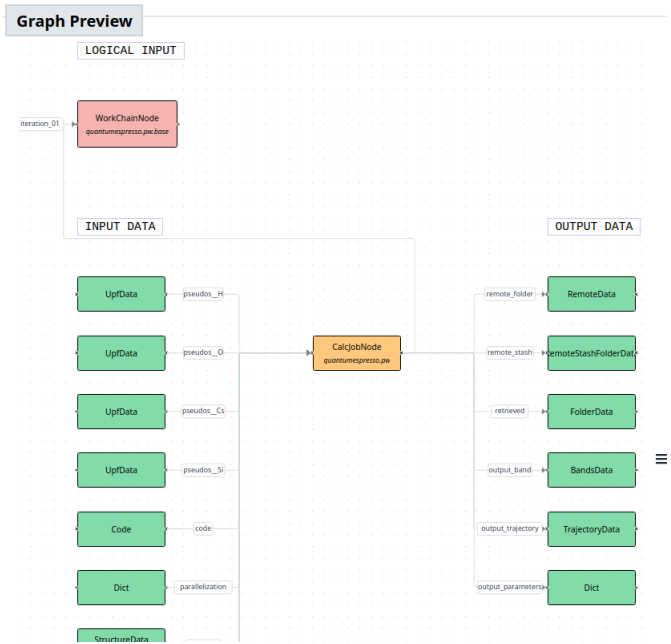
`aiida-common-workflows`: code-agnostic interface for 13 engines

aiida.net/plugin-registry · github.com/aiidateam/aiida-common-workflows

Publishing and browsing



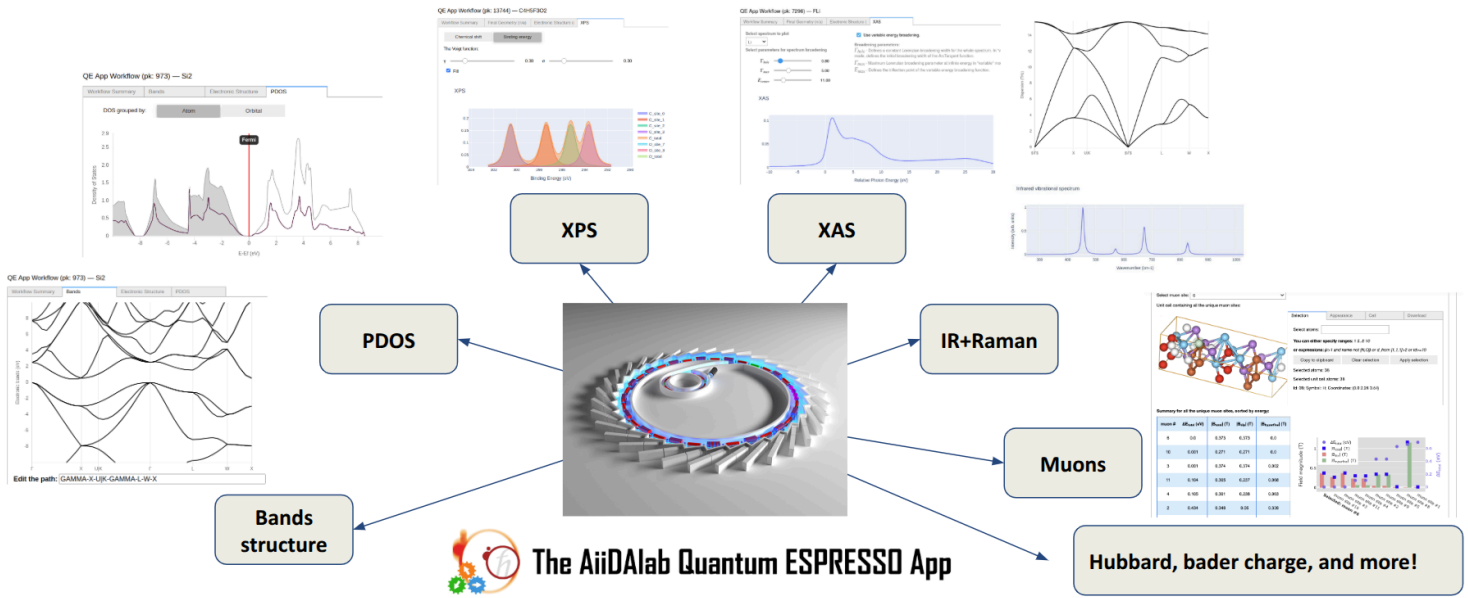
Materials Cloud Archive: 1238 records, each with a DOI



Published provenance, browsable node by node

archive.materialscloud.org · materialscloud.org/explore

AiiDAlab



- Jupyter-based, with an app store; 16 apps registered
- Each panel above a plugin, with its own workchains
- Public demo, institutional JupyterHubs, or local Docker
- `aiida-core` as the engine underneath

aiidalab.net · demo.aiidalab.io

Thanks!

Questions?





aiida.net · aiida.readthedocs.io · github.com/aiidateam/aiida-core · *v3 board* github.com/orgs/aiidateam/projects/21